MAGNUS SIS

Sistema Web de Gestión de Inventario

Modelado lógico · prototipo visual · propuesta de esqueleto Django

INTEGRANTES

- Kevin Julian Sanchez Perez
- Juan Jose Osorio Quitian
- Luis Esteban Medina
- Brayan Steven Sanchez
- Sergio Neira
- Hilver Galindo

Estado: mockups estáticos terminados; el proyecto Django está en etapa inicial. El CRUD, la app de productos y la conexión MySQL aún son trabajo pendiente.

Stack objetivo: Python · Django MVT · HTML/CSS · SQLite local / MySQL objetivo

Fecha: Octubre de 2026

Contenido y alcance

Funciones previstas y estado real del proyecto.

CONTENIDO

1. Modelado: flujos, casos de uso, clases y arquitectura.
2. Prototipado: navegación y seis pantallas de referencia.
3. Esqueleto: estado del repositorio, estructura propuesta, stubs y datos.
4. Resumen del README y tecnologías.

ALCANCE FUNCIONAL PROPUESTO

Aplicación web académica para gestionar productos y categorías: registrar, consultar, buscar, actualizar y eliminar productos; validar datos y representar existencias. Las categorías clasifican productos.

Fuera de alcance: facturación, pagos y reportes complejos.

Mockups: capturas de HTML/CSS estático. Los productos, cifras, precios, fechas y estados son ejemplos de diseño, no datos guardados ni funciones operativas.

Código: la estructura y los stubs de inventario descritos son la propuesta a implementar. En el repositorio actual solo está creado el proyecto base Django, con SQLite.

1. Diagramas lógicos (modelado del sistema)

1.1 Diagrama de flujo - Gestión de Productos

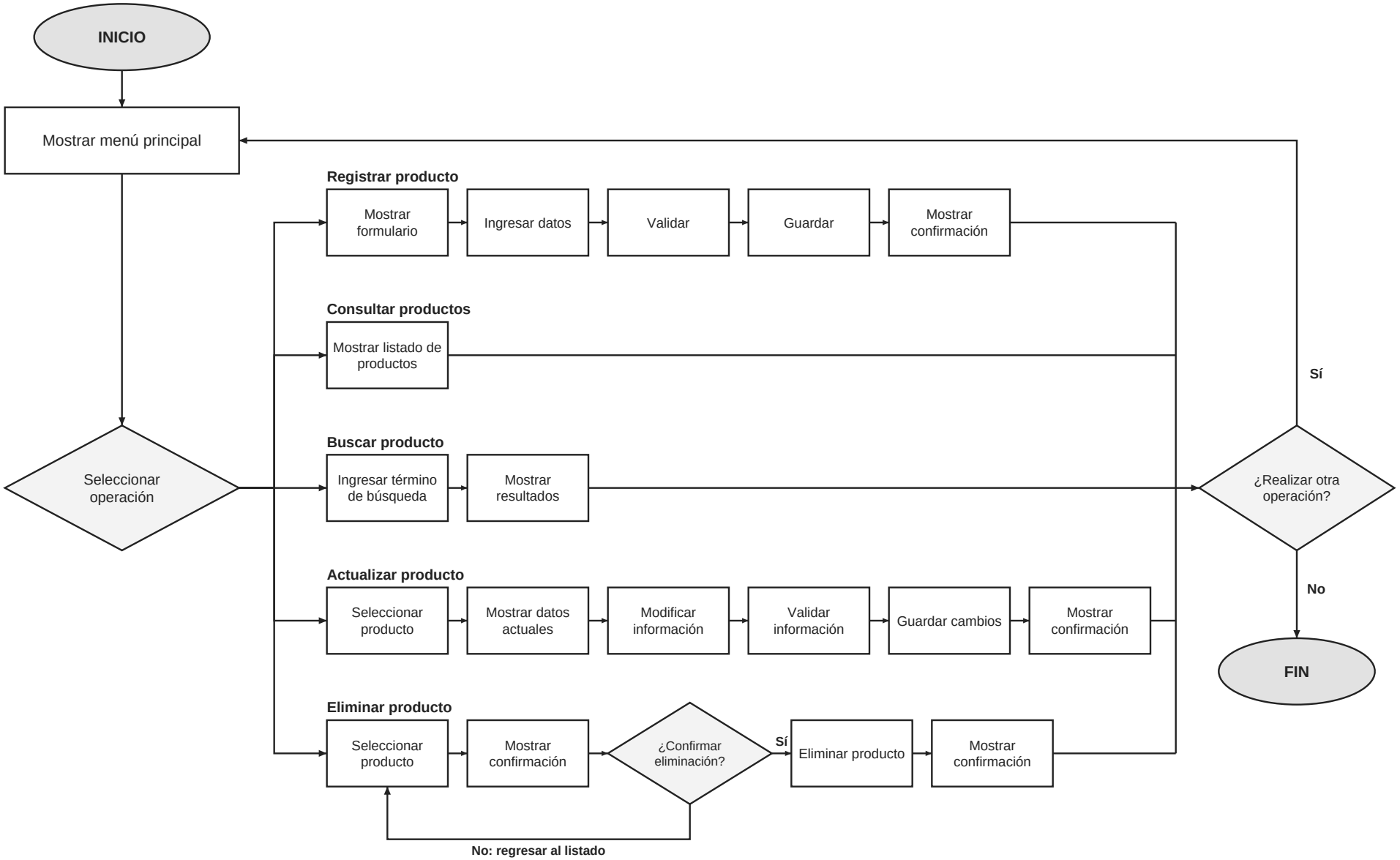


Figura 1. Flujo general: menú, selección de operación y rutas de registrar, consultar, buscar, actualizar y eliminar.

1.2 Diagrama de flujo - Registro de Producto

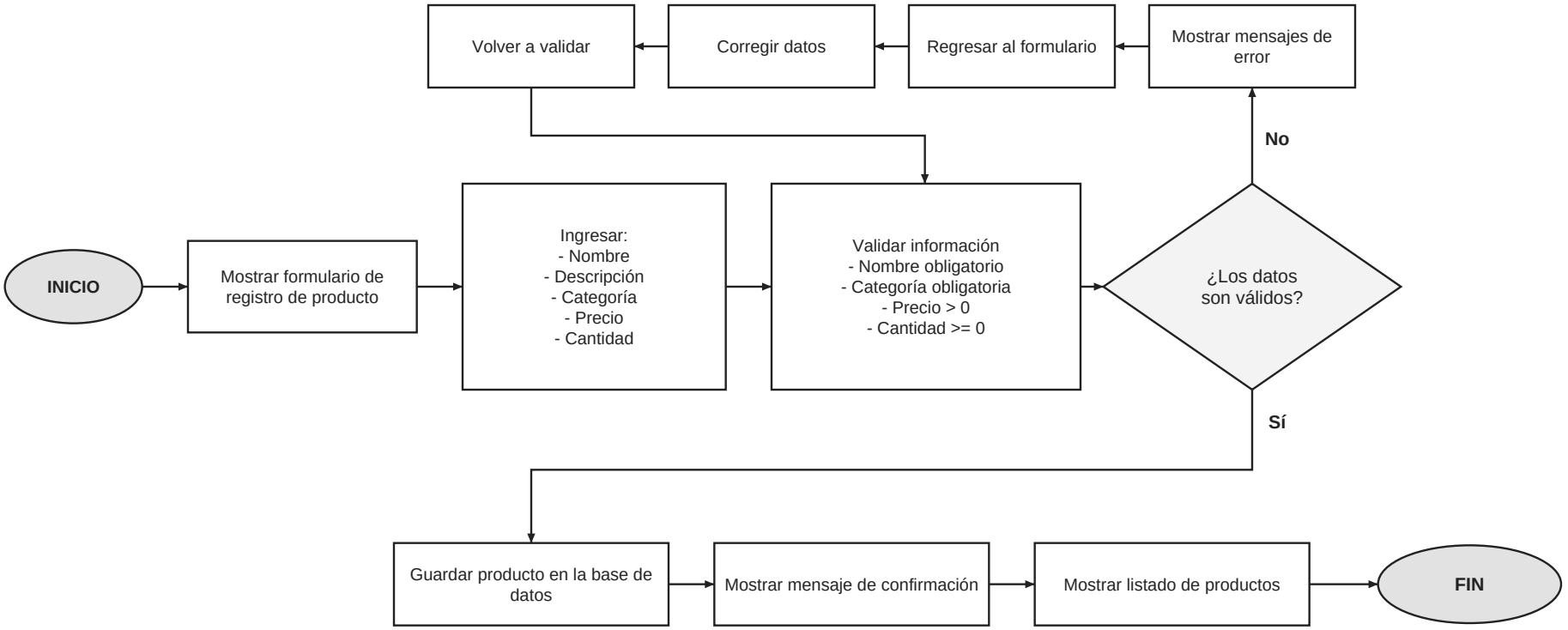
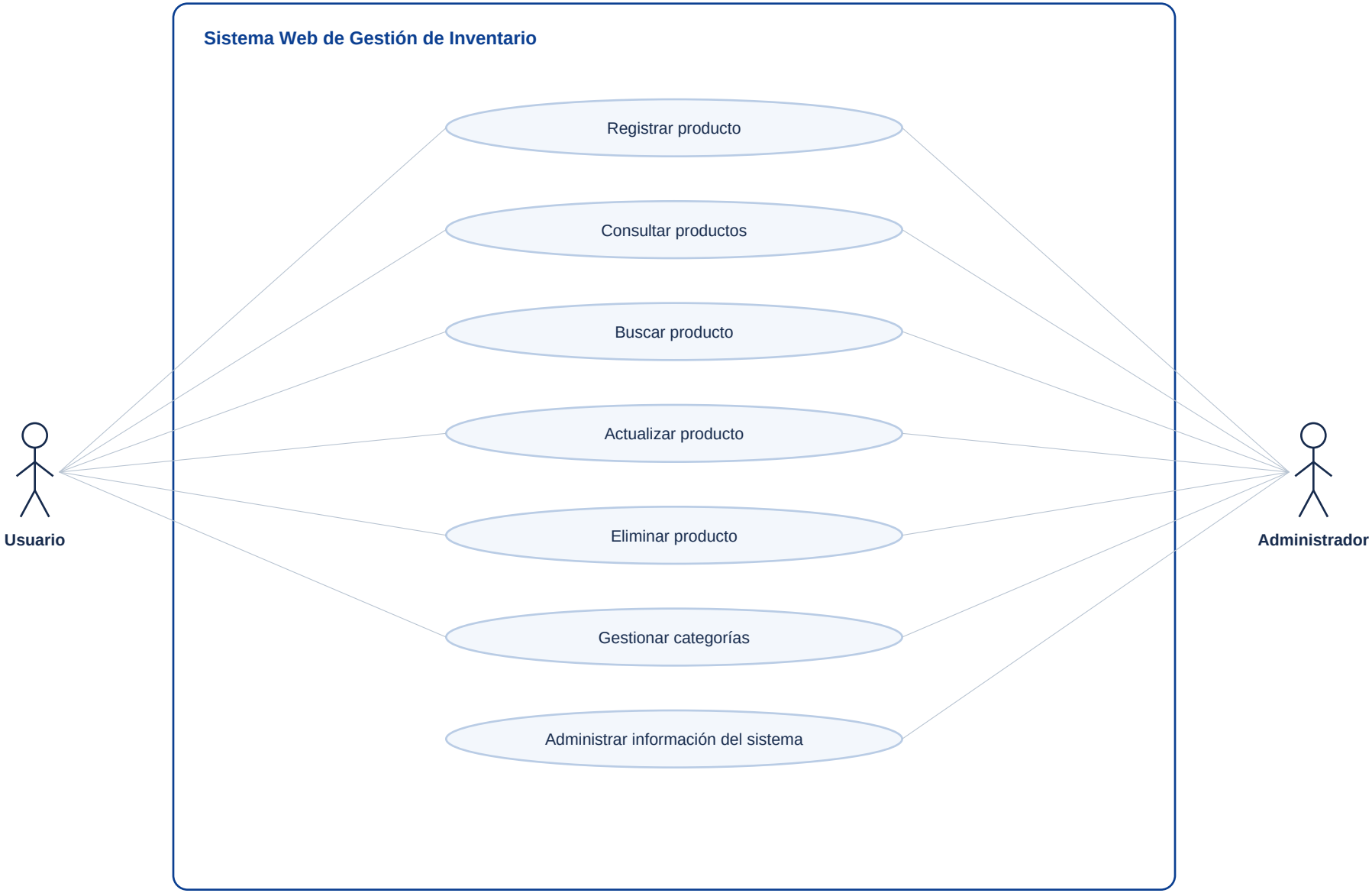


Figura 2. Proceso crítico de validación de entrada con ciclo de corrección. Validaciones: nombre y categoría obligatorios, precio mayor que cero, cantidad mayor o igual a cero.

1.3 Casos de uso

El Administrador puede realizar las operaciones del Usuario y acceder al panel de Django.



Asociaciones simples. Los perfiles y permisos son conceptuales; el mockup muestra un usuario administrador de ejemplo y no implementa autenticación.

Detalle de actores y acciones

Actor	Acciones que puede ejecutar
Usuario	Registrar, consultar, buscar, actualizar y eliminar productos; consultar y gestionar categorías.
Administrador	Realizar las mismas operaciones del Usuario y administrar información desde el panel administrativo de Django.

Límite del sistema: las funciones anteriores pertenecen al sistema de inventario; el panel Django es acceso administrativo. Las asociaciones son simples, sin include ni extend. Los actores y permisos son conceptuales: la autenticación aún no está implementada.

1.4 Diagrama de clases (UML)

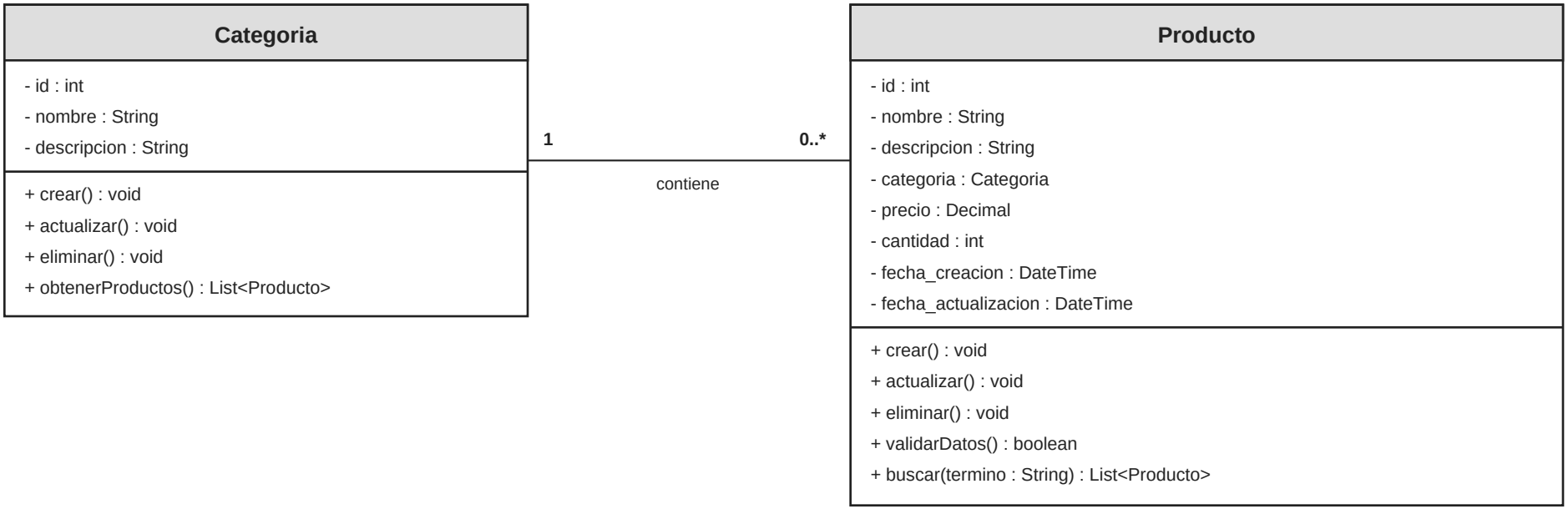


Figura 4. Clases Categoria y Producto con atributos tipados (-), métodos (+) y cardinalidad 1 a 0..*.

Relación: una categoría tiene cero o muchos productos; cada producto pertenece a una sola categoría. El modelo es simple y no requiere herencia ni composición. Los métodos se muestran como stubs propuestos, en coherencia con el apartado 3.3.

1.5 Arquitectura objetivo

El HTML/CSS es un prototipo visual separado del backend Django pendiente.



El ORM, las entidades y la conexión MySQL no están implementados en el proyecto actual.

2. Prototipado y navegación

Seis pantallas HTML/CSS estáticas representan rutas principales previstas.

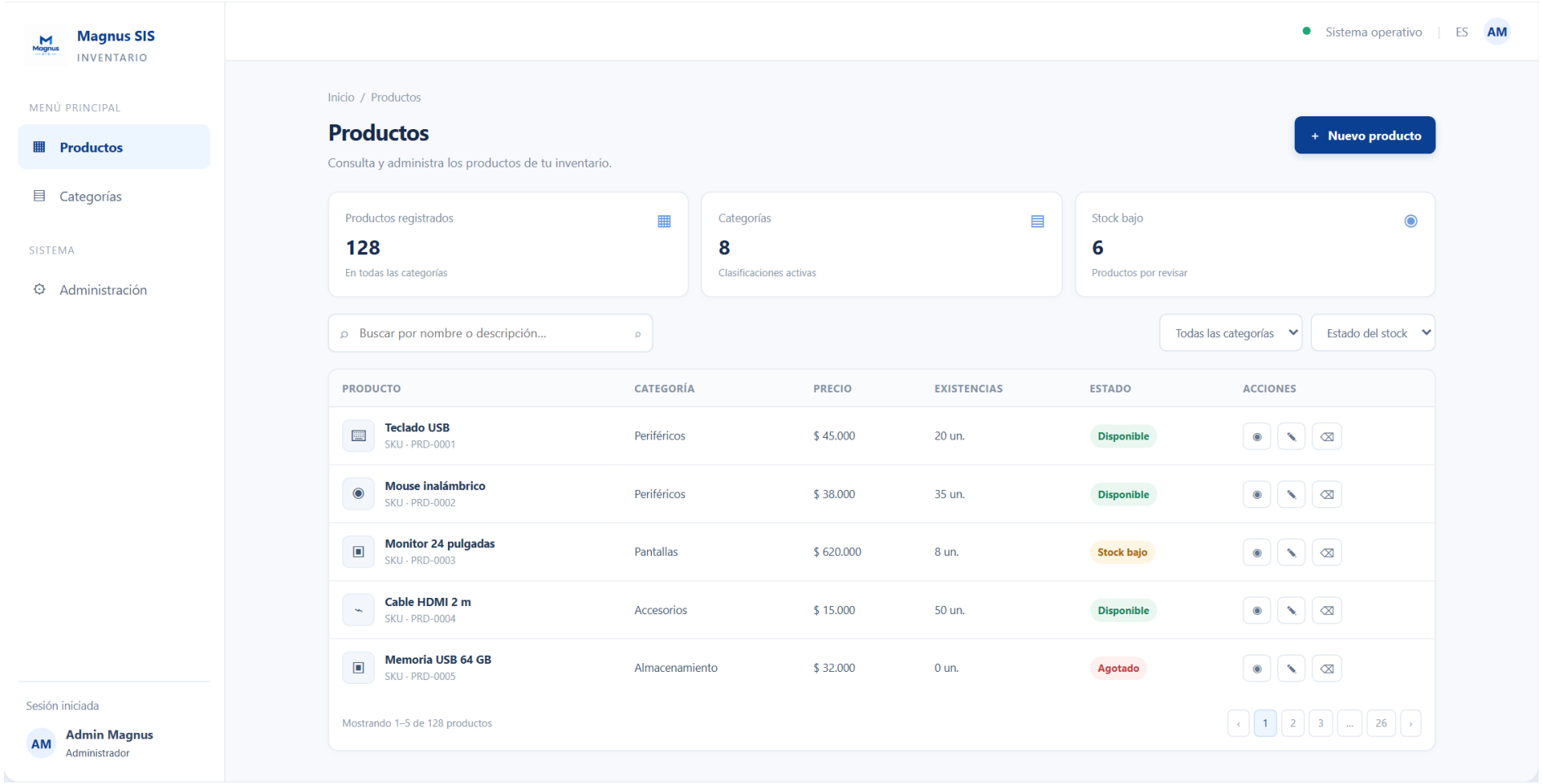
Pantalla	Ruta prevista	Correspondencia
Productos	/	Listado, búsqueda, filtros y stock
Nuevo producto	/productos/nuevo/	Formulario y validación prevista
Detalle	/productos/<id>/	Datos del producto y existencias
Editar / eliminar	/productos/<id>/editar/ · /eliminar/	Formulario y confirmación
Categorías	/categorias/	Listado y acceso visual para crear
Administración	/admin/	Panel Django; fuera del mockup

El prototipo de categorías solo muestra el listado. El formulario para crear o editar categorías queda pendiente. Contadores y ejemplos de productos son ilustrativos.

2.1 Mockup · Inventario de productos

Listado, búsqueda, filtros, existencias y accesos CRUD.

Listado y búsqueda

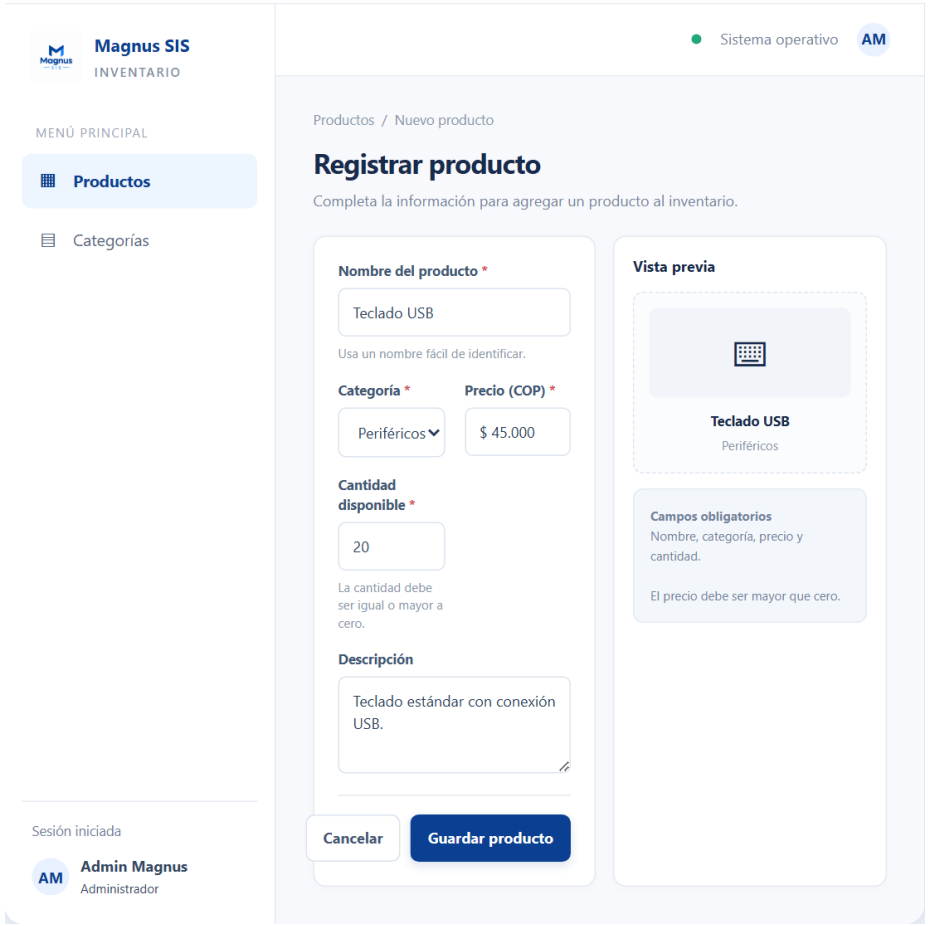


Captura estática. Filas, cifras y estados de stock son datos de ejemplo.

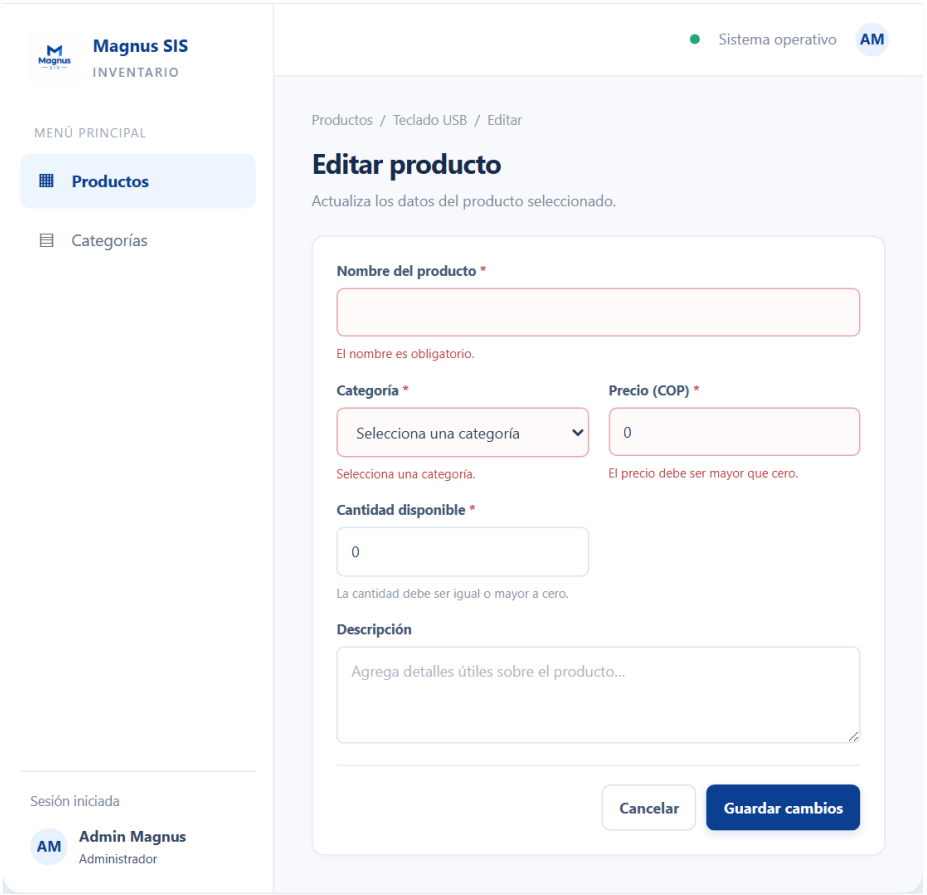
2.2 Mockups · Registro y edición

Formularios y mensajes que corresponden a las validaciones propuestas.

Registrar producto



Editar · mensajes

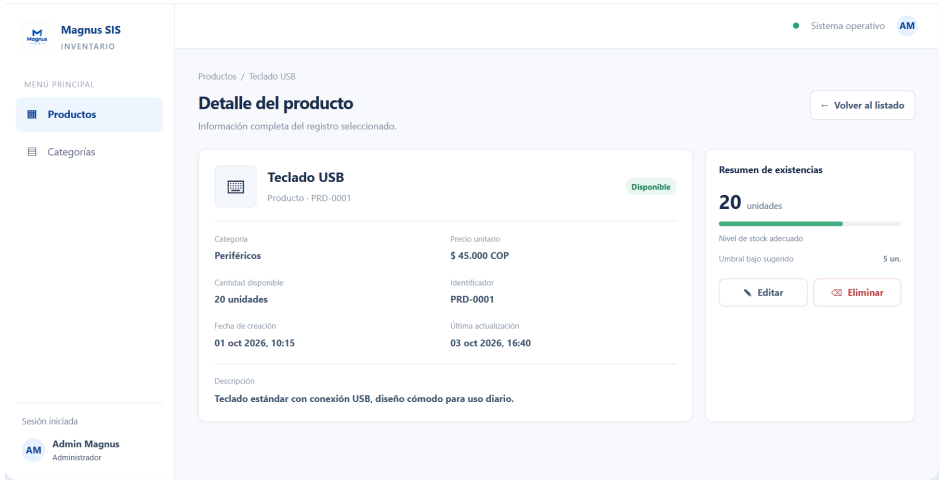


La validación se propone en Django Forms; no está implementada en estas pantallas estáticas.

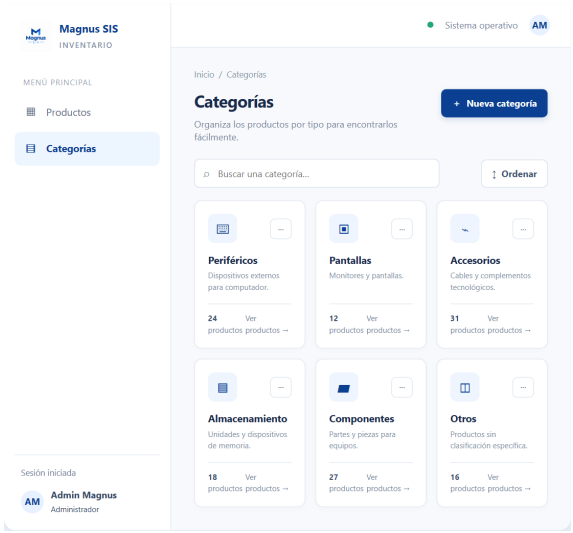
2.3 Mockups · Detalle, categorías y eliminación

Consulta, clasificación y confirmación antes de eliminar.

Detalle



Categorías



Confirmación de eliminación



Los datos son ilustrativos. La pantalla de categorías solo representa el listado; la administración funcional queda pendiente.

3. Estado del código y estructura

Se distingue el contenido presente en el repositorio de la app que se propone implementar.

REPOSITORIO ACTUAL

```
CRUD_MagnusSIS/
■■ README.md
■■ Logotipo corporativo Magnus SIS.png
■■ mockups-inventario.html
■■ mockups-inventario.css
■■ magnus_sis/
  ■■ manage.py
  ■■ magnus_sis/{settings,urls,asgi,wsgi}.py
```

El proyecto tiene /admin/ y SQLite configurado. No existen todavía la app productos, modelos, migraciones, vistas, forms, templates ni rutas CRUD.

ESTRUCTURA OBJETIVO · PENDIENTE

```
magnus_sis/productos/
■■ models.py · Categoria y Producto
■■ forms.py · validaciones
■■ views.py · operaciones CRUD
■■ urls.py · rutas del mockup
■■ admin.py · configuración
■■ migrations/
■■ templates/productos/
  ■■ lista.html
  ■■ detalle.html
  ■■ form.html
  ■■ confirmar_eliminar.html
```

No se afirman `manage.py check` ni pruebas de humo aprobadas: la app y sus pruebas todavía no existen en el repositorio.

Repositorio remoto de GitHub: no se indicó una URL verificable.

3.3 Clases y funciones stub

PROPUESTA · STUB
No integrado ni probado en el repositorio.

productos/models.py

```
"""Modelos del sistema (stubs: sin lógica de negocio)."""
from django.db import models


class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField(blank=True)

    def __str__(self) -> str:
        return self.nombre

    def crear(self) -> None:
        pass

    def actualizar(self) -> None:
        pass

    def eliminar(self) -> None:
        pass

    def obtener_productos(self) -> list:  # UML: obtenerProductos()
        return []


class Producto(models.Model):
    nombre = models.CharField(max_length=150)
    descripcion = models.TextField(blank=True)
    categoria = models.ForeignKey(
        Categoria, on_delete=models.PROTECT, related_name="productos"
    )
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    cantidad = models.PositiveIntegerField(default=0)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    fecha_actualizacion = models.DateTimeField(auto_now=True)

    def __str__(self) -> str:
        return self.nombre

    def crear(self) -> None:
        pass

    def actualizar(self) -> None:
        pass

    def eliminar(self) -> None:
        pass

    def validar_datos(self) -> bool:  # UML: validarDatos()
        return False

    @staticmethod
    def buscar(termino: str) -> list:
        return []
```

PROPUESTA · STUB
No integrado ni probado en el repositorio.

productos/forms.py

```
"""Formularios y validaciones (stubs)."""
from django import forms

from .models import Categoria, Producto


class ProductoForm(forms.ModelForm):
    class Meta:
        model = Producto
        fields = ["nombre", "descripcion", "categoria", "precio", "cantidad"]

    def clean(self) -> dict:
        # Pendiente: nombre y categoría obligatorios, precio > 0, cantidad >= 0
        return super().clean()


class CategoriaForm(forms.ModelForm):
    class Meta:
        model = Categoria
        fields = ["nombre", "descripcion"]
```

productos/views.py

```
"""Vistas del sistema (stubs: devuelven plantillas vacías)."""
from django.http import HttpRequest, HttpResponse
from django.shortcuts import render


def lista_productos(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/lista.html", {})


def buscar_producto(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/lista.html", {})


def detalle_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/detalle.html", {})


def crear_producto(request: HttpRequest) -> HttpResponse:
    return render(request, "productos/form.html", {})


def editar_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/form.html", {})


def eliminar_producto(request: HttpRequest, pk: int) -> HttpResponse:
    return render(request, "productos/confirmar_eliminar.html", {})


def lista_categorias(request: HttpRequest) -> HttpResponse:
    return HttpResponse("Categorías: pendiente de implementar")
```

PROPUESTA · STUB
No integrado ni probado en el repositorio.

productos/urls.py

```
"""Rutas de la aplicación productos."""
from django.urls import path

from . import views

app_name = "productos"

urlpatterns = [
    path("", views.lista_productos, name="lista"),
    path("buscar/", views.buscar_producto, name="buscar"),
    path("productos/nuevo/", views.crear_producto, name="crear"),
    path("productos/<int:pk>/", views.detalle_producto, name="detalle"),
    path("productos/<int:pk>/editar/", views.editar_producto, name="editar"),
    path("productos/<int:pk>/eliminar/", views.eliminar_producto, name="eliminar"),
    path("categorias/", views.lista_categorias, name="categorias"),
]
```

productos/tests.py

```
from django.test import TestCase
from django.urls import reverse


class HumoTests(TestCase):
    def test_lista_responde(self):
        self.assertEqual(self.client.get(reverse("productos:lista")).status_code, 200)

    def test_formulario_responde(self):
        self.assertEqual(self.client.get(reverse("productos:crear")).status_code, 200)
```

3.4 Datos y configuración

Esquema relacional propuesto; SQLite es la única base configurada actualmente.

MODELO DE DATOS OBJETIVO

Categoria: id PK, nombre VARCHAR(100) obligatorio, descripcion TEXT opcional.

Producto: id PK, nombre VARCHAR(150) obligatorio, descripcion TEXT opcional, categoria_id FK obligatoria, precio DECIMAL(10,2) mayor que cero, cantidad entero ≥ 0, fechas de creación y actualización.

Una categoría puede contener cero o muchos productos; cada producto pertenece a una categoría.

CONFIGURACIÓN

Actual: `magnus_sis/settings.py` usa SQLite local. No configura variables de entorno para cambiar a MySQL.

Objetivo: MySQL mediante Django ORM, pendiente de instalar y configurar.

Pendiente: `requirements.txt` y dependencia del conector MySQL.

Los ejemplos de `.gitignore` ampliado y `requirements.txt` del borrador original son propuestas, no archivos presentes en la raíz.

README.md · resumen alineado

Descripción de referencia que refleja el estado real del proyecto.

Sistema Web de Gestión de Inventario

Integrantes

Kevin Julian Sanchez Perez · Juan Jose Osorio Quitian · Luis Esteban Medina
Brayan Steven Sanchez · Sergio Neira · Hilver Galindo

Descripción

Propuesta académica para gestionar productos y categorías. El prototipo HTML/CSS presenta listado, búsqueda, formularios, detalle, categorías y confirmación de eliminación. El CRUD Django con persistencia queda pendiente.

Tecnologías

Prototipo: HTML5 y CSS3. Backend objetivo: Python y Django MVT. Base actual: SQLite; destino previsto: MySQL.

Estado

Existe el proyecto base Django y el mockup estático. La app de inventario, validaciones, migraciones, integración MySQL y pruebas no están implementadas.

El README de la raíz sigue siendo una descripción breve; este resumen en el PDF no afirma que el README ya se haya modificado.

Repositorio: https://github.com/kevinsanchezpe-ops/CRUD_MagnusSIS